What Does the Target Leave Behind? Sparse-Autoencoder Concept Pruning for Text–Tabular Learning

Or Davidovich
Technion – Israel Institute of Technology

October 20, 2026

Abstract

In text–tabular learning, each free-text column is embedded and compressed before a tree model sees it. Joint TAR fine-tunes the encoder on the target so that this compression keeps target-relevant variation. We ask whether one can go further by *removing* concepts: we decompose each column's embeddings with a sparse autoencoder (SAE), judge every concept jointly with its sibling concepts and the table, subtract the concepts that do not help predict the target, and compress what remains. Across the 20 text datasets of MulTaBench, the selection is consistently far better than removing the same number of random concepts ([+x.xxx], [n]/20 datasets), and large shares of the representation ([xx%] of concepts, [xx%] of variance) can be removed at little cost. Yet removal does not improve on Joint TAR ([Δ, p]), even when the amount removed is chosen by nested cross-fitting so that every decision is made on rows the encoder never saw. We argue that this is a property of the problem rather than of the selection rule: after target-aware fine-tuning, what is left to remove carries little variance that the downstream compression would otherwise waste.

1 Introduction

Many real tables mix numbers and categories with free text: job ads, product reviews, wine descriptions. A standard recipe embeds each text cell with a pretrained encoder, reduces it to a few dimensions, and gives the result to a gradient-boosted tree model alongside the other columns [7, 11]. MulTaBench [13] evaluates two variants of this recipe. *Joint Frozen* embeds with a frozen E5 encoder [14] and keeps 30 principal components per column; *Joint TAR* first fine-tunes the encoder on the target, so that the principal components follow the variation that matters for the label.

Both variants keep *everything* the text expresses and let compression decide what survives. A product description, however, talks about many things—materials, shipping, brand tone—of which only a few relate to the price. Principal components follow variance, not relevance, so a frequent but irrelevant theme can occupy components that a weaker relevant theme needs. This suggests an explicit alternative: break the embedding into interpretable concepts, remove those unrelated to the target, and only then compress.

Sparse autoencoders (SAEs) provide such a decomposition: they write an embedding as a sparse, non-negative combination of learned directions that tend to be individually interpretable [3, 5, 6]. We use them as a filter rather than as features. Our contributions are:

- A concept-pruning layer for text–tabular pipelines: per-column SAE, joint sibling-aware relevance testing against shuffled "shadow" concepts, and subtractive map-back into the embedding space (§3).
- A leakage-controlled evaluation on all 20 MulTaBench text datasets, with a random-removal control and a nested cross-fitted variant that makes every pruning decision on rows the encoder never saw (§4).

- A negative but informative result: the method knows *which* concepts matter, but removing the rest does not beat target-aware fine-tuning; we analyse why (§5).

2 Related work

Text in tabular learning. Benchmarks of tables with text fields [11, 13] show that tree ensembles over compressed text embeddings are strong baselines. Target-aware text representations appear both in fine-tuning the encoder before compression (Joint TAR, 13) and in tabular foundation models that attend from target tokens to verbalised cells, such as TabSTAR [1].

Supervised dimensionality reduction. Supervised PCA keeps only features correlated with the outcome before extracting components [2]; partial least squares finds directions of maximal covariance with the target [15]. Our method applies the screening idea in an SAE concept basis instead of raw coordinates, and removes concepts instead of selecting coordinates.

Feature relevance with correlated features. Univariate screening and marginal importance are unreliable when features are redundant [12]. Boruta [10] compares each feature with permuted "shadow" copies inside one model; we extend it to groups of sibling SAE concepts. Out-of-fold estimation and cross-fitting [4, 16] motivate our nested variant.

Sparse autoencoders. SAEs are a central tool of mechanistic interpretability [3, 5]; we use TopK SAEs [6]. Prior work uses SAE concepts to interpret or steer models; we test whether they can *filter* representations for a downstream predictor.

3 Method

Pipeline. For each text column c, let $x \in \mathbb{R}^{384}$ be the (frozen or TAR-tuned) E5 embedding of the string "`passage: c: value`". Joint Frozen and Joint TAR fit PCA-30 on the training rows' x and pass the 30 components to LightGBM [9] together with the table. We insert one step before PCA, fitted on the training rows only.

Concept decomposition. We train a TopK SAE [6] with $L = 2048$ latents and $k = 32$ active per input on the column's training embeddings: $z = \mathrm{TopK}(W_e(x - b))$, $\hat{x} = b + \sum_i z_i d_i$ with unit-norm decoder directions d_i. The SAE is unsupervised. Concepts active on fewer than 20 training rows are never tested and are always kept.

Joint, sibling-aware relevance. Testing concepts one at a time fails in two ways: near-duplicate "sibling" concepts (common in large SAEs) each look dispensable because the other covers for it, and concepts that matter only together with the table or other concepts look irrelevant. We therefore (i) cluster eligible concepts by decoder direction (average-linkage cosine distance < 0.5) into sibling groups; (ii) fit one LightGBM on *[table, all concepts, a shadow copy of every group]*, where a group's shadow is its activations with rows permuted jointly; and (iii) drop a group whose total split gain does not exceed its own shadow's in a majority of three repeats.

Map-back. Dropped concepts are subtracted, keeping the SAE's reconstruction error: $\tilde{x} = x - \sum_{i \in \mathcal{D}} z_i d_i$. PCA-30 is then fitted on $\tilde{x}$, so the downstream model sees exactly as many text features as in Joint Frozen/TAR.

Table 1: Paired comparison with Joint TAR (per-dataset means over 6 folds). Δ: mean difference in AUROC/R^2; p: Wilcoxon signed-rank test over datasets.

[table generated by make_tables.py once the runs finish]

Choosing how much to remove. The rule above fixes the amount implicitly. Two variants choose it from data. *Held-out*: rank groups by harm (variance taken divided by importance measured as the rise in loss when the group is permuted on TAR's early-stopping rows) and pick the removal fraction $f \in \{0, .1, .25, .5, .75\}$ by loss on those rows. *Cross-fitted*: split the training rows into three parts; for each part, fine-tune TAR on the other two, rank concepts there, and evaluate every f on the unseen part with a LightGBM of the final shape (table and all text columns). The f with the lowest total held-out loss is applied to the fold's ordinary TAR. In both variants $f = 0$ (plain TAR) is a candidate, and a larger removal must lower the loss by more than 0.1%.

4 Experimental setup

Data and protocol. All 20 text datasets of MulTaBench (5 binary, 7 multiclass, 8 regression), using the benchmark's own splits (two repeats of 3-fold cross-validation, 6 folds; training sets capped at 10k rows) and metrics: AUROC (one-vs-rest macro for multiclass) and R^2. Joint TAR follows the benchmark recipe: LoRA [8] of rank 16 on the last three E5 layers, a linear head, regression targets binned into 20 quantiles, early stopping on 10% of the training rows. All arms share the fold's split, LightGBM configuration, and—for TAR-based arms—the *same* fine-tuned encoder, so differences isolate the pruning step.

Controls. *Random pruning* removes the same number of eligible concepts, chosen at random, from the same SAE. We also report frozen-embedding variants (relevance pruning and concept pruning without fine-tuning), PLS-30 [15], and out-of-fold stacking of a text-only model's predictions [16].

Leakage control. Every component that uses the target (SAE ranking, shadow tests, amount selection, stacking) is fitted on the training rows of the outer fold only; test rows are never used. Stacking folds keep rows with identical text together, since duplicated texts otherwise leak through the out-of-fold models.

Statistics. Because methods that use the target must be compared with Joint TAR, we test each arm against our own TAR run on the same splits with a two-sided Wilcoxon signed-rank test over per-dataset means; the 6 folds of a dataset are not independent and are only reported as win/loss counts.

5 Results

Removal does not beat TAR. Table 1 summarises all arms against Joint TAR. [Fill from the final table: TAR+concept pruning Δ, datasets won, p. Preliminary (5 datasets): -0.006, 0/5, 2/16 folds.] Choosing the amount on TAR's held-out rows does not help ([Δ]): on roughly a thousand rows, the five candidate amounts differ by less than the selection noise, and the apparent gains do not transfer.

But the selection is far from random. Removing the same number of random concepts from the same SAE is much worse ([Δ vs. TAR+concept pruning], [n]/20 datasets), so the joint test reliably identifies which concepts matter. Table 2 shows how much is removed: [xx%] of the eligible concepts, carrying [xx%] of the embedding variance.

Table 2: How much concept pruning removes, averaged over datasets, folds and text columns.

[table generated by make_tables.py once the runs finish]

Table 3: Nested cross-fitted removal amount (five datasets). Every decision is made on rows the judging encoder never saw.

[table generated by make_tables.py once the runs finish]

Clean decisions do not change the picture. Table 3 reports the cross-fitted variant on five datasets. [Fill: chosen fractions and Δ.]

Why there is no headroom. Three observations point to the same explanation. First, on frozen embeddings, PLS—a supervised *projection*—improves over Joint Frozen ([Δ, p]), while concept pruning followed by PCA does not; the useful signal from the target enters through choosing *directions to keep*, not through deleting concepts. Second, TAR already performs that steering inside the encoder, so the variance that PCA would waste on irrelevant concepts is small after fine-tuning. Third, removing concepts also removes their share of genuinely useful variation that is spread thinly across many concepts; random removal makes this cost visible.

6 Limitations and conclusion

We used a single encoder (E5-small), one SAE size, and LightGBM as the only downstream model. A pilot with TabSTAR [1] as the backbone was infeasible on our hardware (two epochs within its 10-hour budget on one dataset) and is not reported as a result. Held-out rows for choosing the removal amount are few; cross-fitting addresses this at three times the fine-tuning cost.

Concept pruning is a principled, interpretable filter that reliably separates relevant from irrelevant concepts, but after target-aware fine-tuning it finds little left to remove. For practitioners, the lesson is to spend supervision on *what to keep*—fine-tuning or supervised projection—rather than on what to delete.

Code. [Repository URL]. All results can be regenerated with `sae_pilot.py`; tables with `paper/make_tables.py`.

References

[1] Alan Arazi, Eilam Shapira, and Roi Reichart. TabSTAR: A foundation tabular model with semantically target-aware representations. *arXiv preprint arXiv:2505.18125*, 2025.

[2] Eric Bair, Trevor Hastie, Debashis Paul, and Robert Tibshirani. Prediction by supervised principal components. *Journal of the American Statistical Association*, 101(473):119–137, 2006.

[3] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

[4] Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018.

[5] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[6] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

[7] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *NeurIPS Datasets and Benchmarks Track*, 2022.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[9] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 2017.

[10] Miron B. Kursa and Witold R. Rudnicki. Feature selection with the Boruta package. *Journal of Statistical Software*, 36(11):1–13, 2010.

[11] Xingjian Shi, Jonas Mueller, Nick Erickson, Mu Li, and Alexander J. Smola. Benchmarking multimodal AutoML for tabular data with text fields. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[12] Carolin Strobl, Anne-Laure Boulesteix, Thomas Kneib, Thomas Augustin, and Achim Zeileis. Conditional variable importance for random forests. *BMC Bioinformatics*, 9:307, 2008.

[13] TODO: authors – copy from arXiv 2605.10616. MulTaBench: Benchmarking multimodal tabular learning with text and image. *arXiv preprint arXiv:2605.10616*, 2026.

[14] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[15] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58(2):109–130, 2001.

[16] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

A Per-dataset results

Table 4: Mean AUROC (classification) or R^2 (regression) over 6 folds, LightGBM. Best per row in bold. †: fewer than 6 folds.

[table generated by make_tables.py once the runs finish]